AI Meets Formal Verification: Practice and Prospects

Liao Zhang

Beneficial AI Foundation

November 6, 2026

Overview

1. Background: Beneficial AI Foundation and Signal Shot
2. SPQR and Dalek
   - 2.1 Practice
   - 2.2 Lessons
3. Auto-Specifying Dalek
   - 3.1 Practice
   - 3.2 Lessons
4. Prospects

About Me

- Research Scientist at the Beneficial AI Foundation.
- Working on AI for formal verification since 2018.
- PhD from the University of Innsbruck, Austria.
- Research focus: AI for mathematics and software development.

Selected Research

AI for Term Rewriting

Pioneering AI methods for term rewriting, solving the most problems in the annual Confluence Competition.



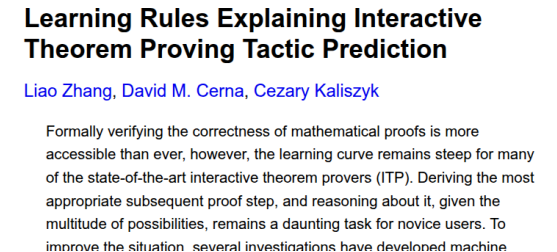
Automated Strategy Invention for Confluence of Term Rewrite Systems

Liao Zhang, Fabian Mitterwallner, Jan Jakubuv, Cezary Kaliszyk

Term rewriting plays a crucial role in software verification and compiler optimization. With dozens of highly parameterizable techniques developed to prove various system properties, automatic term rewriting tools work in an extensive parameter space. This complexity exceeds human capacity for parameter selection, motivating

Interpretable Tactic Prediction

Learning explicit rules that explain tactic predictions in interactive theorem proving.



Learning Rules Explaining Interactive Theorem Proving Tactic Prediction

Liao Zhang, David M. Cerna, Cezary Kaliszyk

Formally verifying the correctness of mathematical proofs is more accessible than ever, however, the learning curve remains steep for many of the state-of-the-art interactive theorem provers (ITP). Deriving the most appropriate subsequent proof step, and reasoning about it, given the multitude of possibilities, remains a daunting task for novice users. To improve the situation, several investigations have developed machine

Selected Research

Evaluating AI Agents on Property-Based Testing

From documentation to semantic invariants and input-generation strategies that reveal bugs.

PBT-Bench: Benchmarking AI Agents on Property-Based Testing

Lucas Jing, Xinqi Wang, Liao Zhang, Simon S. Du

Existing code benchmarks measure whether an agent can produce any test that reproduces a known bug, or whether it can produce a patch that fixes a described issue. Neither isolates the distinct skill of property-based testing: deriving a semantic invariant from documentation, and then constructing an input-generation strategy precise enough to make a random search reveal the violation. We introduce PBT-Bench, a benchmark of 100 curated property-based

Roadmap

1. **Background: Beneficial AI Foundation and Signal Shot**
2. **SPQR and Dalek**
3. **Auto-Specifying Dalek**
4. **Prospects**

Beneficial AI Foundation

Mission

Keep AI beneficial for humanity through technical AI safety research.

- Founded by Max Tegmark.
- A central question: how can we make AI systems safer and more reliable?
- One approach: use AI to generate programs, formally verify their correctness and safety properties, and use the verified programs to carry out user tasks.

Connection to practice: cryptographic software provides concrete, demanding verification tasks.



Max Tegmark

What Is Signal?

- **Signal the app:** a messaging app for private conversations, including text messages, group chats, and voice/video calls.
- **End-to-end encryption:** messages are encrypted on the sender's device and decrypted on the recipient's device. The server relays ciphertext without access to the decryption keys.
- **Signal Protocol:** the cryptographic protocol underlying these encrypted conversations.



Why verification? Establish that the protocol and its implementation satisfy their intended security properties.

Signal Shot: A Shared Verification Challenge

Launched in April 2026: a collaborative initiative to formally verify the Signal protocol and application using Lean.

- **Technical ambition:** work toward machine-checked guarantees across the protocol stack and its implementation.
- **Engineering ambition:** develop reusable tools for protocol design and continuous verification as software changes.
- **Community ambition:** bring together Lean, cryptography, software verification, and AI-assisted theorem proving.
- **Concrete workstreams:** the initiative links verification repositories for libsignal, PQXDH, and SPQR.

Roadmap

1. Background: Beneficial AI Foundation and Signal Shot

2. SPQR and Dalek
   2.1 Practice
   2.2 Lessons

3. Auto-Specifying Dalek

4. Prospects

SPQR and Dalek

Practice

SPQR: A Key-Updating Component of Signal

SPQR = Sparse Post-Quantum Ratchet

- **The protocol:** a component of Signal that updates encryption keys during a conversation, designed to resist quantum attacks.
- **The code:** Signal also provides a Rust library implementing SPQR. This is the software we study.
- **Why "sparse"?** The data needed for a fresh post-quantum key update is spread across multiple messages.

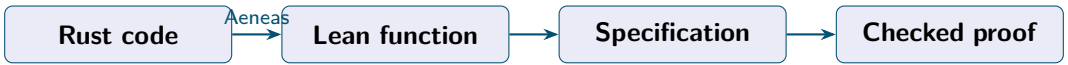
Intuition: Alice and Bob keep updating their keys as they exchange messages.

What We Verify

Using Lean, we prove properties of the **Rust code**: for example, that data is encoded correctly and key-update steps follow their specifications. This is implementation correctness, not a proof of all of Signal's security.

SPQR: From Rust Code to Checked Proofs

Target: functional correctness of Signal's Sparse Post-Quantum Ratchet.



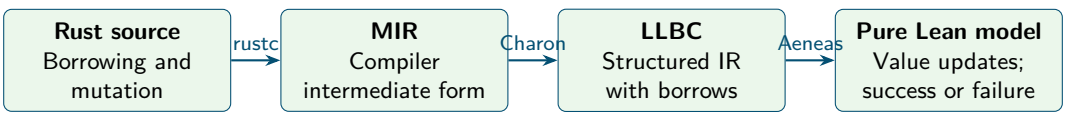
1. **Translate:** use Aeneas to turn the Rust implementation into Lean functions.
2. **Specify:** state what those functions should do, including their assumptions.
3. **Prove:** construct proofs that the translated functions satisfy these specifications; Lean checks the proofs.

The Key Distinction

Translation gives us code we can reason about in Lean. Specifications and proofs establish the properties we want to verify.

Aeneas: Translating Rust into Lean

Inside the translation: Rust control flow becomes a functional model



- **Translation:** Aeneas symbolically executes LLBC, tracking values, borrows, and loans. Mutation becomes explicit value updates in pure functions.
- **Semantic foundation:** formal results relate the borrow-based semantics to a heap-and-address model for the formalized language, not the entire Rust toolchain.

SPQR Example: A Byte Comparison Loop

Compare two byte slices by accumulating differences with bitwise XOR and OR.

Rust: Mutation and a `for` Loop

```
fn compare(lhs: &[u8], rhs: &[u8]) -> u8 {
    let mut r: u8 = 0;
    for i in 0..lhs.len() {
        r |= lhs[i] ^ rhs[i];
    }
    is_non_zero(r)
}
```

`is_non_zero(r)` returns 0 if $r = 0$, and 1 otherwise.

Lean: Explicit State and `Result`

```
def compare (lhs rhs : Slice U8) : Result U8 := do
  let n := Slice.len lhs
  let r ← compare_loop
    { start := 0#usize, «end» := n } lhs rhs 0#u8
  is_non_zero r
```

Loop body at index `i` (names shortened):

```
let a ← Slice.index_usize lhs i
let b ← Slice.index_usize rhs i
let d ← lift (a ^^^ b)
let r1 ← lift (r ||| d)
ok (cont (iter1, r1))
```

Rust construct	**Lean representation**
`&[u8]`	`Slice U8`: a borrowed byte slice becomes a value.
`let mut r; for`	`loop` threads state `(iter, r)`; `cont` continues, `done` exits.
`lhs[i]`	`Slice.index_usize`: a checked access in `Result`; failure propagates.

SPQR Example: Specification and Proof

Specification and Top-Level Proof (`compare_spec`)

```
theorem compare_spec (lhs rhs : Slice U8)
    (hlen : lhs.length = rhs.length) :
    compare lhs rhs {{ (result : U8) =>
      result.val =
        if (∀ (j : Nat), j < lhs.length → lhs.val[j]! = rhs.val[j]!)
        then 0 else 1 }} := by
  unfold compare
  step*
  grind
```

Meaning: equal-length inputs $\Rightarrow$ successful execution; return 0 for equal slices and 1 otherwise.

Loop invariant: $r = 0 \iff \forall j < i,\ lhs[j] = rhs[j]$.

Initialize $i = 0$, $r = 0$: the empty prefixes match.

Preserve OR accumulates XOR differences; bit-vector lemmas establish the invariant at $i + 1$.

Finish $n - i$ decreases; equal lengths ensure valid accesses. At $i = n$, use `is_non_zero`.

`step*` applies helper specifications; `grind` closes the logical goal. The theorem establishes functional behavior, not constant-time execution.

SPQR: Reviewing Long Specifications

Example: `encaps1` returns ciphertext, intermediate state, and a shared secret.

Contract Sketch (Simplified from Lean)

```
Assume: header length >= 64
  fill_bytes succeeds
Guarantee: there exists one message m
  of 32 bytes such that
  ct1 = ct1List(header, m)
  es  = stateOut(header, m)
  ss  = ssOut(header, m)
Lengths:
  ct1: 960, es: 2080, ss: 32
```

What must a reviewer check?

- All three outputs use the *same* message `m`.
- The named model functions describe the intended bytes, not just their sizes.
- Input conditions match actual callers.
- Success of `fill_bytes` does not establish uniform randomness.

Difficulty: Lean checks the proof of the statement; humans must still check that the statement captures the intended behavior.

SPQR: A True Specification Can Be Too Weak

Example: garbage collection of stored message keys. Old keys should be removed while the required live keys are preserved.

A Plausible but Insufficient Specification

"Every key remaining after cleanup is valid."

Counterexample A: Delete Everything

The output is empty, so every remaining key is valid. The statement holds even though all live keys are lost.

Counterexample B: Duplicate a Live Key

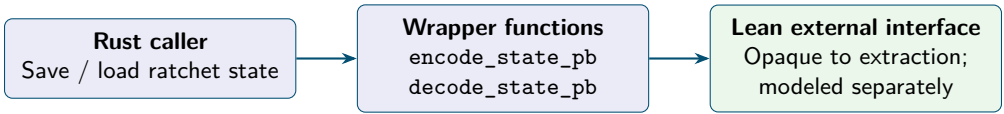
Keep one live key twice and lose another. Each remaining entry can still be valid.

Review obligation: specify which entries survive and preserve their multiplicity, rather than only checking each output entry in isolation.

Lesson: a completed proof cannot repair a missing requirement.

SPQR: Code Outside the Translation

Example: serializing the ratchet state with Protobuf (`prost`). The project's extraction patch records problems translating the generated `Message` implementations.



- The extraction patch routes calls through wrappers and marks them `opaque`.
- This lets the surrounding code be translated without translating the `prost` implementation.
- The extraction configuration also excludes platform-specific SIMD and some generated trait implementations.

The Extra Verification Obligation

A proof about a handwritten Lean model still needs justification that the external Rust code behaves like that model.

SPQR: Reviewing External Assumptions

Example: the external `libcrux_hmac::hmac` call.

Two Different Declarations in Lean

```
noncomputable opaque libcrux_hmac.hmac : ...
axiom libcrux_hmac.hmac_sha256_tag32_spec ...
```

Signatures shortened: the external names a function; the axiom assumes its behavior.

What This Axiom Actually Says

For key and data lengths $\leq$ `u32::MAX`, the SHA-256 HMAC call requesting 32 bytes succeeds and returns 32 bytes.

- **Bounds:** external length conversions to `u32` can panic.
- **Scope:** length and success, not the HMAC value or cryptographic security.
- **Trust:** Lean uses this assumption; downstream proofs depend on it.

Dalek: What It Does and What We Verify

- **What is it?** `curve25519-dalek` is a Rust library for group operations on Curve25519 (Edwards and Montgomery forms) and Ristretto.
- **What does it compute?** Point addition $P + Q$, scalar multiplication $[s]P$, and conversions between points, scalars, and bytes.
- **Where is it used?** These operations support key agreement, digital signatures, and zero-knowledge proofs; protocol libraries build on them.

Our Verification Project

Use Aeneas and Lean to verify selected functions in the 64-bit serial backend: field arithmetic modulo $2^{255} - 19$, byte encodings, and curve operations.

Example guarantee: for a valid point P and scalar bytes with the top bit clear, scalar multiplication succeeds and returns a valid point Q with

$$\text{toPoint}(Q) = [\text{decode}(s)]\,\text{toPoint}(P).$$

This connects the concrete Rust computation to the mathematical group operation.

SPQR and Dalek

Lessons

AI Is Now Highly Capable

- A year ago, AI struggled with challenging proofs in my workflow.
- Proofs involving loop invariants were particularly difficult.
- In my recent projects, AI has completed every proof task I have attempted with it.
- These include Lean proofs exceeding 1,000 lines.
- Most AI-generated specifications I have reviewed have been correct.

Simple Prompts Often Suffice

- **MCP (Model Context Protocol)** is an open standard for connecting AI applications to external tools and data.
- An MCP server exposes tools that an AI agent can discover and call, then returns their results to the agent.
- **Lean MCP integrations**, such as `lean-lsp-mcp`, connect the agent to Lean's language server and search tools:
  - Inspect the current proof goal and local hypotheses.
  - Read compiler errors, warnings, types, and documentation.
  - Search for relevant lemmas and definitions.
  - Try tactics or code snippets and inspect Lean's feedback.
- **Workflow:** inspect a goal $\rightarrow$ propose a proof step $\rightarrow$ check with Lean $\rightarrow$ revise using feedback.

Simple Prompts Often Suffice

- Recent improvements in AI capabilities have simplified my proof workflow.
- I examined Claude Code's behavior during Lean proof development.
- In the sessions I examined, it used neither skills nor MCP tools to construct Lean proofs.
- Direct file access and compilation were sufficient in these sessions.
- Structured proof views can help inspect individual goals.
- I also need project-wide information, such as the number of remaining `sorry` placeholders.
- In this workflow, elaborate prompts have offered limited additional benefit.
- My most useful instruction: "Decompose the proof and compile after each completed step."

Example: Prove2Me and Fermat's Last Theorem

- **Prove2Me** supported the Claude-agent project that formalized Fermat's Last Theorem in Lean.
- Agents can communicate with the remote platform through ordinary **HTTP requests and JSON responses**, using tools such as `curl`.
- Write and compile a proof locally, submit it for remote checking, then use the returned verdict and errors to revise it.

A Simple Remote Verification Loop

`POST /api/v1/verify`: upload `solution.lean`; receive a submission ID.

`GET /api/v1/verify?submission_id=...`: poll for the verdict and errors.

- The platform tracks subtheorem dependencies so agents can work on separate parts of a large proof.
- **Takeaway:** a simple request–response interface can connect capable agents to a large-scale formal verification workflow.

Clarity of AI-Generated Explanations

- Despite its capabilities, AI still requires human guidance.
- Clear, comprehensible explanations are essential for effective human oversight.
- AI systems tend to produce lengthy responses.
- My most frequent instruction is: "Please shorten your response while preserving precision and clarity."
- AI models should communicate their results in language that users can readily understand.

Roadmap

1. Background: Beneficial AI Foundation and Signal Shot

2. SPQR and Dalek

3. Auto-Specifying Dalek
   3.1 Practice
   3.2 Lessons

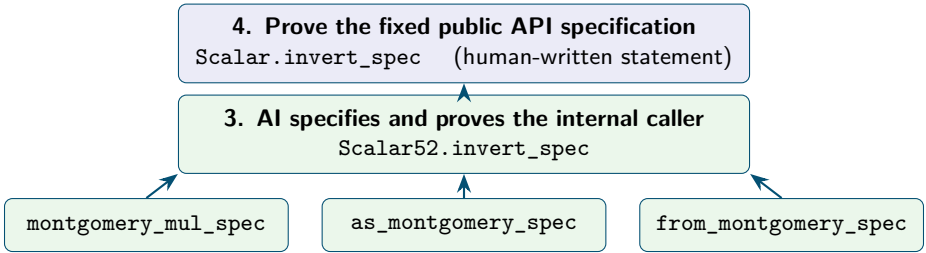
4. Prospects

Auto-Specifying Dalek

Practice

Auto-Specifying Dalek: Bottom-Up Proofs

1. A human defines the public API specification, which remains fixed.
The implementation remains unchanged; AI writes internal specifications and proofs.

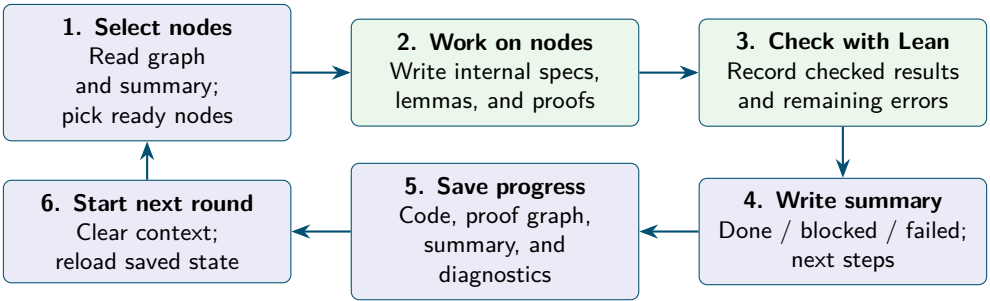


2. AI writes and proves lower-level specs (`Scalar52` helpers)

Arrows: proved dependencies support the caller above; continue from the leaves to the API.
Feedback: if an internal spec is too weak, revise it and recheck affected proofs.
Final check: Lean proofs and project checks pass; the supplied API statement remains unchanged.

Harness: Node Selection and Fresh Rounds

Main agent: decide which proof nodes to work on using the dependency graph.



Preserving Progress Across Sessions

Each round starts fresh, but retains the fixed API goal, code, dependency graph, and concise summaries. Repeat until the API proof and final checks pass.

Round design shown here: reset after every summary. The harness also supports session continuation.

Auto-Specifying Dalek

Lessons

A Simple Harness Can Be Effective

- Use a simple workflow to coordinate proof development.
- Assign a central agent to determine which proof node to expand next.
- Determine which proof nodes can be addressed within the current iteration.
- Record progress after each iteration.
- Reset the working context periodically, retaining concise progress summaries.

Decompose Difficult Proofs and Check Incrementally

- Initial attempts to prove `to_bytes` were unsuccessful.
- The `to_bytes` implementation exceeds 700 lines of code, and proof attempts frequently time out.
- The key improvement was to decompose complex theorems into simpler lemmas.
- Make a small change, then check it with Lean.

Cross-Model Review for Specification Writing

- AI can generate specifications, but these specifications may contain errors.
- Complement manual review with independent reviews by different AI models.
- This approach is particularly useful for less critical specifications.
- Examples include helper lemmas and functions outside the public API.
- In my experience, having different AI models independently check the same specification is highly effective at finding errors and omissions.

Roadmap

1. Background: Beneficial AI Foundation and Signal Shot
2. SPQR and Dalek
3. Auto-Specifying Dalek
4. **Prospects**

Future Directions for AI-Assisted Formal Verification

- Efficient inference.
- Understandable formal verification.

Why Efficient Inference Matters

- AI can now automate substantial parts of proof development and program verification.
- Long proof searches can require substantial token usage and computation.
- For practical adoption, verification costs must be justified by the value of the assurance gained.
- Reducing inference costs makes verification feasible for more software projects.

How to Improve Efficiency?

- **Specialized small models and efficient inference**
  - Vision already benefits from compact architectures, distillation, pruning, and quantization.
  - **MobileNet:** 4,866M $\rightarrow$ 569M multiply-adds (**88% less computation**); ImageNet accuracy 71.7% $\rightarrow$ 70.6%.
- **Opportunity for math and code:** apply these ideas to specialized proof/code models; optimize **cost per verified solution**.

How to Improve Efficiency?

- **Better verification frameworks**
  - Decompose a goal into small subgoals.
  - Let automated theorem provers solve routine subgoals.
  - Improve the translation from Rust to Lean.
    - Generate Lean functions that express Rust semantics clearly.
    - Structure the translated code to simplify proof obligations.

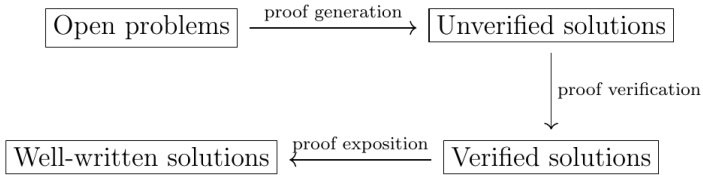
Understandable Formal Verification

- AI can generate proofs faster than people can read them.
- A checked proof still needs a clear explanation of its main ideas.

Terence Tao, August 2026

MATHEMATICS IN THE AGE OF AI

TERENCE TAO



Proof explanations are essential for human understanding.

Understandable Formal Verification

- Use proof assistants with **Controlled Natural Language (CNL)**.
- CNL resembles mathematical English, with restricted grammar and precise formal meaning.
- The system translates proof steps into logical obligations; automated provers check them.

Illustrative CNL Sentences and Their Logical Meaning

Every prime number is greater than one.	$\forall p \in \mathbb{N},\ \mathrm{Prime}(p) \Rightarrow p > 1$
There exists an even prime number.	$\exists p \in \mathbb{N},\ \mathrm{Prime}(p) \wedge \mathrm{Even}(p)$

- **Examples:** Naproche and Visored.
- **Key idea:** human-readable proofs with machine-checkable structure.

We Are Hiring

Beneficial AI Foundation



Scan to explore open positions

beneficialaifoundation.org/jobs